CURATED STUDY ROUTES AND PRACTICAL PROTOCOLS

Learning Materials

Read | Predict | Experiment | Explain



Original manga illustration: a story analogy. Formal labeled diagrams inside the book carry the technical definitions.

A companion to all 25 chapters: 12 resource cards, three study routes, offline alternatives and an honest record of what has been reviewed.

First-year learning edition | 6 November 2026

K.I.V.A.S. Learning Lab | Foundations of Computer Science & Artificial Intelligence

Inside this book

Choose the matching chapter first. One purposeful resource session is more useful than opening every link.

Choose a resource for a learning need 3

Algorithms: from instructions to a justified search 4

Data structures: operations before containers 5

Connectionist AI: arithmetic before claims 6

RES-01 | Hello Algo: binary search and Bubble Sort 7

RES-02 | NIST Dictionary of Algorithms and Data Structures 8

RES-03 | VisuAlgo: linked lists, stacks and queues 9

RES-04 | Data Structure Visualizations 10

RES-05 | DSA Lab 11

RES-06 | But what is a Neural Network? 12

RES-07 | Machine Learning Crash Course 13

RES-08 | TensorFlow Playground 14

RES-09 | MIT 6.006: Introduction to Algorithms, Spring 2020 15

RES-10 | Learn the Basics 16

RES-11 | Open Data Structures: AdjacencyMatrix 17

RES-12 | K.I.V.A.S. local binary-search lab and video library 18

Map the existing eight-video library 19

Experiment record: preserve what actually happened 20

Resource audit and fallback decisions 21

Choose a resource for a learning need

Use the guide for the complete core explanation. Add a visual source when you need another representation, an interactive tool when you want to test a prediction, and a university course or coding tutorial only when you are ready for the extra prerequisites. Time estimates in this companion are planning suggestions, not measured learner results.

Need	Start here	What to produce
Trace search or sorting	RES-01 and 02	A step-by-step table
Inspect links or removal order	RES-03 or 04	Before/after drawings
Compare an alternative explanation	RES-05	A translated convention
Understand a network picture	RES-06	A fully labeled small network
Understand splits and metrics	RES-07	A justified evaluation plan
Explore training settings	RES-08	A controlled observation log
Extend algorithm rigor	RES-09	One claim with assumptions
Move toward implementation	RES-10	An annotated workflow
Encode graph edges	RES-11	A checked adjacency matrix
Work offline	RES-12	A trace or retrieval response

What verification means here

Public pages and selected documentation were retrieved on 6 November 2026. The resource cards distinguish those reviews from live interactive execution. External links and interfaces can change. No experiment result, account access or full video viewing is claimed unless explicitly documented. The paper alternatives remain complete when a site is unavailable.

Algorithms: from instructions to a justified search

Start with the listed chapters. Adjust the session length to your pace. End each session with an output another learner could inspect. If the evidence is weak, repeat a smaller case before adding a new resource.

Session 1 | 35-45 min

Read: ALG-CH01 to 03

Write and debug a three-step return procedure. Trace an accumulator with an empty case.

Ready when: A partner can follow the instructions without supplying missing actions.

Session 2 | 30-40 min

Read: ALG-CH04 and 05; RES-01

Compare a first-match scan with binary search on a sorted row.

Ready when: Your trace names bounds, comparison outcomes and a valid missing result.

Session 3 | 30-40 min

Read: ALG-CH06 and 07; RES-02

Trace Bubble Sort and compare lookup budgets including preparation.

Ready when: You distinguish swaps from comparisons and explain the cost-model assumptions.

Retrieval | 20 min

Read: Test Bank ALG items

Answer five mixed items without opening the solutions; correct one error using a new example.

Ready when: You can explain why your original reasoning failed.

SPACING AND RETRIEVAL

Return a day later without the answer key. Reconstruct one trace, then compare. After several days, change the input and explain whether the same rule still applies. Record the mistake you corrected, not just the minutes spent.

Data structures: operations before containers

Start with the listed chapters. Adjust the session length to your pace. End each session with an output another learner could inspect. If the evidence is weak, repeat a smaller case before adding a new resource.

Session 1 | 35-45 min

Read: DS-CH01 to 03

Build records, then compare an array edit with a linked splice.

Ready when: You distinguish a durable ID, a position and the cost of locating a node.

Session 2 | 30-40 min

Read: DS-CH04 and 05; RES-03 or 04

Run the same arrival sequence through a stack and queue.

Ready when: Your returned values match LIFO or FIFO, including the empty case.

Session 3 | 40-50 min

Read: DS-CH06 to 08; RES-11

Draw a tree and graph, then choose storage for a small library system.

Ready when: You separate hierarchy, connections, route costs and service order.

Retrieval | 25 min

Read: Test Bank DS items

Complete one diagram task and one design response before checking the rubric.

Ready when: A second reader can reconstruct your operation order.

SPACING AND RETRIEVAL

Return a day later without the answer key. Reconstruct one trace, then compare. After several days, change the input and explain whether the same rule still applies. Record the mistake you corrected, not just the minutes spent.

Connectionist AI: arithmetic before claims

Start with the listed chapters. Adjust the session length to your pace. End each session with an output another learner could inspect. If the evidence is weak, repeat a smaller case before adding a new resource.

Session 1 | 35-45 min

Read: AI-CH01 to 04; RES-06

Define features and compute weighted sums under two activations.

Ready when: You separate input, parameter, sum, activation and output meaning.

Session 2 | 40-50 min

Read: AI-CH05 to 07

Trace the 2-2-1 network and one gradient update.

Ready when: Every intermediate value is labeled and training differs from inference.

Session 3 | 35-45 min

Read: AI-CH08; RES-07 or 08

Calculate errors and design a data split matching a future-use claim.

Ready when: You state what the score supports and what remains unknown.

Session 4 | 30-40 min

Read: AI-CH09 and 10

Draw a service pipeline with an optional model and a human correction route.

Ready when: Your design remains useful when personalization is declined.

SPACING AND RETRIEVAL

Return a day later without the answer key. Reconstruct one trace, then compare. After several days, change the input and explain whether the same rule still applies. Record the mistake you corrected, not just the minutes spent.

RES-01 | Hello Algo: binary search and Bubble Sort

Hello Algo contributors
Core companion | 25-35 minutes
ALG-CH03 to ALG-CH07; DS-CH02

https://www.hello-algo.com/en/chapter_searching/binary_search/

Why this resource: A visual textbook provides a second representation of search intervals and algorithm steps. Use it after the guide so that unfamiliar code syntax does not obscure the idea. The English binary-search chapter distinguishes interval conventions; the sorting chapter illustrates adjacent exchanges.

Access and review status: The public English pages were read on 6 November 2026. Reading required no account in the fetched pages. Code language tabs and animations are supplementary; their complete live behavior was not tested in this review.

Set up: Open section 10.1 and choose the closed-interval version to compare with K.I.V.A.S. Keep zero-based indices and inclusive low/high bounds. Copy [3, 6, 9, 12, 15, 18, 21] and target 13 onto paper. For sorting, follow the linked Bubble Sort chapter in the site menu.

Predict and try: Predict the midpoint before reading the next explanation. Trace the missing target, then change only the target to 15. Next use[4, 3, 1, 2] and predict one complete ascending Bubble Sort pass. Pause before each adjacent comparison rather than watching the final arrangement alone.

Record: For search, record low, high, mid, value and excluded side. For sorting, record each compared pair, whether it swaps and the resulting row. Label the interval convention used by each source.

Interpret: Our search convention inspects 12, 18, 15 in both trials;13 ends with empty bounds and 15 is found. The first sorting pass ends[3, 1, 2, 4]. If a source uses another midpoint or interval convention, reconcile its valid trace before calling it wrong.

Limits: Matching a published example demonstrates a trace, not a proof for every input. Different valid variants may use different intermediate bounds. Growth claims assume the relevant access and comparison costs.

Offline alternative: Use seven numbered cards and two paper boundary markers. Hide crossed-out candidates. For sorting, slide only adjacent cards. The complete algorithms and feedback are in ALG-CH05 and ALG-CH06.

RES-02 | NIST Dictionary of Algorithms and Data Structures

Paul E. Black / NIST
Precision check | 15-20 minutes
ALG-CH05 to ALG-CH07

https://xlinux.nist.gov/dads/HTML/binarySearch.html

Why this resource: Short formal entries help bridge an intuitive story and a precise algorithm definition. Read binary search and the companion Bubble Sort entry after you can already trace both on paper.

Access and review status: Both public entry pages were retrieved on 6 November 2026. They are compact reference pages, not complete beginner courses. No account was needed to read the retrieved content.

Set up: Open binary search, then https://xlinux.nist.gov/dads/HTML/bubblesort.html. Keep the guide beside the definitions. Underline the input condition for search and the operation that distinguishes Bubble Sort.

Predict and try: Translate each entry into a three-sentence explanation for a classmate. Add one counterexample to an omitted condition: unsorted input for binary search, or a single quiet adjacent comparison mistaken for a full quiet pass.

Record: Write the condition, one trace that satisfies it, and one failure caused by removing it. For the midpoint note, record that the guide uses the lower integer midpoint; fixed-width implementation overflow is an optional extension.

Interpret: Binary search depends on sorted order under its comparison key. Bubble Sort uses adjacent comparisons and repeat passes. On[1, 3, 2], the first pair needs no swap but the row is still not sorted.

Limits: A reference definition is concise and may omit classroom details such as response format or duplicate policy. Supply those in your own contract. The entries do not measure performance on your device.

Offline alternative: Rewrite the guide algorithms on one sheet, cover them, then reconstruct the precondition and stop condition from memory. Test your reconstruction on empty, one-item and duplicate-containing rows.

RES-03 | VisuAlgo: linked lists, stacks and queues

Steven Halim and the VisuAlgo team, National University of Singapore
Core visual lab | 30-40 minutes
DS-CH03 to DS-CH05

https://visualgo.net/en/list

Why this resource: This combined visualizer lets you compare sequence links with restricted stack and queue operations. It is most useful for observing which references or endpoints change, then explaining why the logical order is preserved.

Access and review status: The list page was read on 6 November 2026. It documents exploration and e-Lecture modes and may choose a related mode when loaded. It also describes login-dependent shortcuts and training features. Public reading was available; account-specific exercises were not tested.

Set up: Use the list page and select the intended mode from its menu. Begin with a small example of three items; if the page starts in e-Lecture mode, follow its on-page instructions to enter exploration. Keep a paper record of the actual initial values because defaults can vary.

Predict and try: For a linked list, predict a search and insertion before stepping the animation. Then compare stack push A, push B, pop with queue enqueue A, enqueue B, dequeue. If the interface accepts only numbers, substitute 11 for A and 22 for B consistently.

Record: Record initial order, operation, returned item, final order and changed links. Separate a node search from local rewiring. Note the mode selected and whether the interface exposes each operation you expected.

Interpret: The stack removes B/22 first; the queue removes A/11 first. A safe linked insertion retains the former successor. Animation speed does not change the algorithmic promise.

Limits: A visualizer can hide memory-management details and may use a different implementation from your drawing. Do not infer actual constant-time behavior from a fast animation or assume login-only features are available to classmates.

Offline alternative: Use labeled cards with yarn or drawn arrows. Keep one card marked head. For stacks, expose only the top; for queues, mark front and rear. Record every step using the DS chapter labs.

RES-04 | Data Structure Visualizations

David Galles, University of San Francisco
Alternative visual lab | 25-35 minutes
ALG-CH06; DS-CH04 to DS-CH07

https://www.cs.usfca.edu/~galles/visualization/Algorithms.html

Why this resource: The public index links array and linked implementations of stacks and queues, comparison sorting, search trees and graph traversals. It is useful for comparing the same interface implemented using different storage.

Access and review status: The index and its named links were read on 6 November 2026. This review did not execute every linked visualization. Some older implementation families are explicitly listed as Java-only; choose the current browser-linked stack or queue examples from the index.

Set up: Open Stack: Array Implementation and Stack: Linked List Implementation in separate views. Use the same short input sequence in both, recording the values actually accepted by the interface. Repeat with the two queue implementations if time permits.

Predict and try: Predict the returned item for pushes 11, 22, pop, 33, pop. Compare visible storage changes between implementations. For queues, use enqueue 11, 22, dequeue, 33, dequeue. Explain what stays the same even when the pictures differ.

Record: Keep two columns: interface result and representation change. Count logical operations separately from animated frames. Add an empty-operation test and record the interface response rather than assuming one.

Interpret: Both stack implementations should return 22 then 33 under the stated sequence; both queues should return 11 then 22. The storage pictures can differ while the behavior remains equivalent.

Limits: Shared output on one sequence is limited evidence. Add boundary cases and preserve the actual tool response. A browser animation is an educational model, not a performance benchmark.

Offline alternative: Draw both an array and a linked chain for the same sequence. In the array track top or front/size. In the chain track head and any tail reference. Compare final logical order with the guide solution.

RES-05 | DSA Lab

DSA Lab project
Optional second explanation | 20-30 minutes
ALG-CH03 to ALG-CH07; DS-CH01 to DS-CH08

https://www.dsalab.dev/en

Why this resource: The English index lists fundamentals, linear structures, trees, graphs and sorting with step-by-step material. Use one matching topic to compare explanations rather than browsing advanced structures before the foundations are secure.

Access and review status: The public English index was read on 6 November 2026. It lists visualizations, code, exercises and assessments. Individual activity controls and any account-dependent completion tracking were not tested; verify them on your classroom device before assigning a session.

Set up: Choose Stack or Queue from Linear structures, or Bubble Sort from sorting. Start with exactly one topic already completed in K.I.V.A.S. Ignore optional language-specific code until you can explain the operation in ordinary words.

Predict and try: Write your expected result first. Compare the source explanation with a four-item hand trace. Find one difference in notation, operation naming or boundary policy and translate it into the guide's convention.

Record: Record topic URL, date, initial values, operation sequence and the notation difference. If a tool control is unavailable, note the obstacle and finish the paper version instead of recording an imagined run.

Interpret: Equivalent operations should preserve the same logical order once notation and policies are aligned. A real disagreement deserves a small reproducible example and inspection of both specifications.

Limits: The large index includes material beyond this course. Breadth is not a reason to add advanced topics to the required learning route. Index availability does not establish that every activity works on every device.

Offline alternative: Use the corresponding chapter lab and exchange traces with a partner. One learner follows stack rules while the other follows queue rules on identical arrival cards, then they explain the difference.

RES-06 | But what is a Neural Network?

Grant Sanderson / 3Blue1Brown
Core visual explanation | 25-40 minutes
AI-CH02 to AI-CH06

https://www.3blue1brown.com/lessons/neural-networks/

Why this resource: The lesson introduces a network through handwritten-digit recognition. Its diagrams offer an intuitive view of activations, weights and layers. The accompanying text lets you pause and reread without depending only on video playback.

Access and review status: The official lesson page and text adaptation were read on 6 November 2026. The page identifies Grant Sanderson and the original lesson date 5 October 2017. This review did not watch the full video or test all embedded interactions.

Set up: Read or watch the opening explanation and stop at the first layered network. Keep the guide's much smaller 2-2-1 network on another sheet. Do not copy the large diagram's parameters into the small example.

Predict and try: Identify inputs, hidden activations, weights and output interpretation. Pause before an explanation and predict what one connection does. Then calculate the K.I.V.A.S. two-input example from scratch to connect the visual intuition to exact arithmetic.

Record: Write three terms in your own words and label each on the small network. Record the lesson section or actual playback timestamp you used; this companion does not invent timestamps for unviewed footage.

Interpret: A pixel or feature is an input value; a weight controls a contribution; a hidden activation is computed. The small guide network gives Y=2.5 for inputs 2, 1 with its stated parameters, independently of the lesson's different network.

Limits: A visually persuasive analogy does not establish that every hidden unit corresponds to a human concept. Digit recognition performance does not demonstrate book-preference accuracy or general intelligence.

Offline alternative: Use AI-CH02, AI-CH05 and AI-CH06. Draw the small network and place a written number beside every stage. Ask a partner to reproduce the result using only your labels.

RES-07 | Machine Learning Crash Course

Google for Developers
Core selective reading | 30-45 minutes per session
AI-CH05 to AI-CH10

https://developers.google.com/machine-learning/crash-course

Why this resource: The official course has focused modules for neural networks, classification, data splits and generalization. Select the module that addresses your current mistake rather than treating the whole course as a beginner prerequisite.

Access and review status: The course index, Nodes and hidden layers, and Dividing the original dataset pages were read on 6 November 2026. The public pages were accessible. Embedded exercise behavior and optional coding environments were not fully tested.

Set up: For architecture, open Neural networks: Nodes and hidden layers. For evaluation, open Datasets: Dividing the original dataset. Start with the training/validation/test explanation after AI-CH08. Optional Python exercises may require additional preparation.

Predict and try: Write a role card for each data split. Sort six actions into the roles: fit weights, choose learning rate, choose epoch, estimate final performance, inspect a tuning failure, and rerun a fixed final evaluation. Discuss why repeated test-guided design changes compromise independence.

Record: Record action, chosen split and reason. For the hidden-layer lesson, label weighted sum and activation separately. If using an embedded exercise, record the actual starting weights because its page describes randomized initialization.

Interpret: Fitting uses training data; settings and checkpoint choices use validation; final assessment uses appropriate held-out test data after choices are fixed. A test repeatedly used for selection has become part of development.

Limits: A clean split is necessary evidence, not a guarantee of future performance. Related examples, small samples and shifting populations can still make results misleading. The course's larger mathematical scope is optional.

Offline alternative: Use the fictional learning-curve table and reader-group split in AI-CH08. Choose the smallest supplied validation loss and explain why the final test must remain separate from that choice.

RES-08 | TensorFlow Playground

TensorFlow Playground project
Exploratory lab | 35-50 minutes
AI-CH03 to AI-CH08

https://playground.tensorflow.org/

Why this resource: A browser neural-network sandbox makes changing features, layers and training settings visible. It is valuable for observing patterns across controlled trials, while the guide supplies exact small calculations that the animation cannot replace.

Access and review status: The public page was read on 6 November 2026. It exposes learning rate, activation, regularization, data ratio, noise, batch size, features and training/test loss. No model experiment is claimed as completed here. Check the current interface before a class session.

Set up: Suggested starting setup: classification, circle-pattern data, x1 and x2 features only, one hidden layer with four units, Tanh activation, learning rate 0.03, noise 0, batch size 10, 70% training data, and no regularization. Record any setting the current interface cannot reproduce.

Predict and try: Reset and train the baseline to a chosen checkpoint such as 200 epochs. Record it, then repeat from reset with learning rate 0.3, holding the other settings fixed. Repeat each setup three times if time allows. Resetting may change initialization; record that limitation.

Record: Record all settings, epoch, training loss, test loss and a sketch of the decision boundary. Keep actual observations separate from your prediction. Note whether data were regenerated and whether the initialization was controlled.

Interpret: Different rates can produce different progress or instability, but no exact loss is promised. A more complex boundary is not automatically better. If you repeatedly choose settings using the displayed test loss, that display is serving a validation role in your experiment.

Limits: Random initialization and data generation can confound one-run comparisons. The sandbox demonstrates a toy two-dimensional task, not real-world reliability. Do not report a winning setting as universal.

Offline alternative: Use the one-weight experiment in AI-CH07. Starting w=1, x=2, target 4, gradient-8 gives loss 0.16 after rate 0.1 and 196 after rate 1.0. These are deterministic supplied arithmetic examples, not claimed Playground observations.

RES-09 | MIT 6.006: Introduction to Algorithms, Spring 2020

Erik Demaine, Jason Ku and Justin Solomon / MIT OpenCourseWare
Optional university extension | 45-60 minutes
After ALG-CH07 and DS-CH08

https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/

Why this resource: A full university course offers lecture notes, lecture videos, practice problems and assignments. It is a useful next step when you can already justify simple algorithms and want more formal analysis.

Access and review status: The official Spring 2020 course page and its resource categories were read on 6 November 2026. This is the edition linked by the supplied resource PDF. Full lectures were not watched and assignments were not executed in this review.

Set up: Choose one introductory data-structure or sorting item from the course resource index. Read its prerequisites and notation first. Set a modest objective: extract one specification, one data-structure choice and one cost argument.

Predict and try: Translate a selected example into the guide's everyday language. Identify what the course assumes that K.I.V.A.S. has not yet taught. Work a small instance on paper before following a proof or implementation.

Record: Record the exact lecture or note title, link, and the point where you needed an extra prerequisite. Write one claim with its assumptions rather than copying a theorem without understanding it.

Interpret: You should be able to connect at least one formal idea to a familiar operation such as indexed access, comparison or traversal. Difficulty with an advanced proof identifies a learning step; it does not erase your progress in the beginner guide.

Limits: This course is more demanding than the required first-pass learning route. Do not use its programming assignments as an unannounced prerequisite for completing the K.I.V.A.S. paper activities.

Offline alternative: Return to the relevant chapter's cost model. Write a precondition, an invariant and a termination reason for a small search or loop. Use the Test Bank reasoning tasks as the bridge to formal work.

RES-10 | Learn the Basics

PyTorch documentation contributors
Optional coding bridge | 60-90 minutes initially
After AI-CH05 to AI-CH08

https://docs.pytorch.org/tutorials/beginner/basics/intro.html

Why this resource: The official tutorial organizes a machine-learning workflow around data, model construction, automatic differentiation and optimization. It connects the paper concepts to code after the learner has enough Python preparation.

Access and review status: The official introduction page was read on 6 November 2026. Code execution, package installation and notebook environments were not tested for this edition. Follow the current official setup instructions; do not assume a version-specific environment from this book.

Set up: Begin with the introduction and prerequisites. If Python variables, functions and arrays are unfamiliar, postpone execution and read the workflow map only. Use a trusted local environment or the official notebook route described by the tutorial when prepared.

Predict and try: Locate the code responsible for a forward calculation, loss calculation, gradient calculation and parameter update. Annotate those roles before changing code. If you run it, first reproduce the official baseline and preserve the actual output.

Record: Record tutorial URL, software versions, environment, seed if controlled, and the exact change. Keep a copy of the error or result. Never label a successful import as a completed training experiment.

Interpret: The conceptual roles should match the guide: inference produces outputs, loss compares them to targets, gradients describe sensitivity, and an optimizer applies updates. Exact numerical outcomes depend on the actual setup.

Limits: This is optional implementation work, not a condition for understanding the book. Installing software or obtaining one successful run does not demonstrate model suitability or reproducibility across environments.

Offline alternative: Execute the tiny training update by hand in AI-CH07 and label each arithmetic step with its role. Draw arrows that distinguish forward values from backward gradient information.

RES-11 | Open Data Structures: AdjacencyMatrix

Pat Morin
Focused reference | 20-25 minutes
DS-CH07

https://opendatastructures.org/ods-python/12_1_AdjacencyMatrix_Repres.html

Why this resource: A matrix representation turns graph connections into entries that can be inspected directly. This reference is useful after you understand vertices and edges and want a precise bridge from a drawing to stored data.

Access and review status: The public section was read on 6 November 2026. It contains mathematical notation and implementation details beyond the core activity. Focus first on the definition of an entry and the illustrated matrix.

Set up: Draw the guide campus graph with two-way edges LC, CB, LB, BG. Fix row and column order[L, C, B, G]. Use 1 for an edge and 0 for no edge, ignoring travel weights for this adjacency exercise.

Predict and try: Fill all 16 cells. CloseCB and update only the necessary entries. Explain why an undirected edge changes two cells. Then distinguish this 0/1 representation from a matrix that stores travel times.

Record: Record row/column order, diagonal convention and whether the graph is directed. For a weighted version, define how no edge differs from a genuine zero-cost edge rather than silently using an ambiguous value.

Interpret: The undirected 0/1 matrix is symmetric. ClosingCB changes(C, B) and(B, C) from 1 to 0. It does not change LC, LB or BG.

Limits: An adjacency matrix represents relationships; it does not by itself choose a shortest route. Storage choices and traversal algorithms answer different questions.

Offline alternative: Draw a4-by-4 grid and use counters at connected pairs. Remove the two counters for CB. Compare the remaining graph with the route-closure exercise in DS-CH07.

RES-12 | K.I.V.A.S. local binary-search lab and video library

K.I.V.A.S. project; AI-assisted production
Offline companion | 20-35 minutes
ALG-CH01 to ALG-CH07; DS-CH01 to DS-CH07; AI-CH02 to AI-CH08

Why this resource: The project already contains a local browser search lab and eight explanatory videos. Reusing these can support retrieval practice without adding another account or replacing the newly expanded books.

Access and review status: The lab file and prior verification record exist. The video-library index, metadata and source-coverage record were inspected; this edition does not claim a fresh full viewing of all eight videos. The videos follow an older chapter map, so use the topic mapping in this companion.

Set up: Open the included Binary_Search_Lab.html in a browser after extracting the package. For videos, open Computation_Explained/index.html in the existing project. The large MP4 files are kept in that library rather than duplicated into this book package.

Predict and try: Before advancing a search, predict the midpoint and retained side. For a video, pause at a worked example, hide the next step and complete it yourself. Compare the video's notation with the current chapter conventions.

Record: Record target, input row, inspection history and result. For a video, record the actual title, timestamp, prediction and correction. Note any difference caused by its older chapter structure.

Interpret: The guide's standard 15 search inspects 12, 18, 15. A useful viewing session ends with an independent trace or explanation, not merely a checked watch box.

Limits: The existing library includes optional topics beyond this beginner edition. Metadata review is not a fresh content or playback audit. Test the chosen file on the classroom device before relying on it.

Offline alternative: All essential tasks are reproduced in the books. Numbered cards reproduce binary search; the AI chapter tables reproduce the small forward and training calculations.

Map the existing eight-video library

The project's Computation_Explained library already contains eight videos. The table maps their recorded topics to the new books. It is based on the library metadata and source-coverage record, not a fresh full viewing. The older videos include some optional advanced material and use different chapter numbers.

Video	Current chapter match	Approx. length
1. Understanding Algorithms	ALG-CH01 to 03	6m 46s
2. Searching Algorithms	ALG-CH04 and 05	7m 10s
3. Sorting Algorithms and Big O Notation	ALG-CH06 and 07	7m 10s
4. Introduction to Data Structures	DS-CH01 to 05	7m 26s
5. Advanced Data Structures	DS-CH06 and 07	7m 02s
6. Connectionist AI and Artificial Neurons	AI-CH02 to 04	7m 40s
7. Neural Network Architecture	AI-CH05 and 06	8m 12s
8. Neural Network Training, Applications and Limitations	AI-CH07 to 10	8m 13s

Viewing protocol

Before playback, write one question you want the video to answer. Pause at an example, record the actual timestamp, and predict the next state. Afterward, close the video and solve a changed example from the current guide. If an advanced derivation appears before its prerequisites, mark it optional and return to the core arithmetic.

Access

Open Computation_Explained/index.html in the existing project. Videos, transcripts and captions remain there. They are not duplicated into this edition's archive. The included Binary_Search_Lab.html is independent of those large media files.

Experiment record: preserve what actually happened

An experiment log separates predictions, settings and observations. Complete one row per run. Do not copy the expected tendency into the observed-results field when a tool did not run.

Field	Record
Question and chapter	_____
Resource URL and date	_____
Device or environment	_____
Starting data and all settings	_____
Variable changed	_____
Prediction before running	_____
Actual result or access obstacle	_____
Screenshot or saved evidence location	_____
Alternative explanation or confound	_____
Conclusion and limit	_____
Follow-up paper case	_____

Example of an honest conclusion

Three runs with the chosen rate showed lower observed training loss at the selected checkpoint than three runs with the comparison rate. Initialization was not controlled, so this small exercise does not isolate the rate perfectly. I need more controlled repetitions before making a stronger comparison. This is a model of how to report results, not a claim that these runs were performed.

Resource audit and fallback decisions

The original companion named OpenDSA. Attempts to retrieve its public site during this review timed out, and one attempted textbook path returned 404. Those access failures do not establish that the project is unavailable to every reader. It is retained as a candidate for the team to recheck, but this edition does not depend on it. Use Hello Algo, VisuAlgo, the USF visualizations or the paper labs for the affected activities.

The original DSA Lab link was recovered from the PDF annotation and its English index was retrieved. Its individual activities remain marked as unexecuted. MIT's Spring 2020 edition is used here to match the original companion link; the earlier 2011 course was also reachable but is not mixed into this study route.

Before assigning a resource	Evidence to record
Open on a learner's ordinary device	Date, network and access result
Check required controls	Visible names and working operation
Check account requirement	Public versus login-only features
Try the exact example	Starting values and observed result
Check accessibility needs	Text/caption/paper alternative
Check fit to course	Chapter objective and prerequisite
Record a decision	Use, adapt, substitute or defer

Credits and limits

External creators are credited on each resource card. Descriptions and original practice protocols are written for this edition; no external media is redistributed. AI assisted curation and production. The companion is a teaching recommendation, not a claim of endorsement by the listed organizations. Independent learner trials and classroom-device checks remain for the team.